

Basis Pengetahuan Modular OMNI-RAG: Eksperimen Chunker, Indexer, dan Knowledge Graph Multidomain

Moh Fairuz Alauddin Yahya
Program Studi Informatika, Institut Teknologi Bandung
Bandung, Indonesia
13522057

Abstract—Sebuah proyek Capstone beranggotakan empat orang membangun OMNI-RAG, yaitu platform *Retrieval-Augmented Generation* (RAG) multidomain untuk chatbot asisten hukum peraturan perundang-undangan Indonesia. Paper ini memusatkan kontribusi penulis pada jalur offline pembentukan basis pengetahuan: *chunker*, *indexer*, dan pembangun *knowledge graph* yang dipasang sebagai *plugin*. Kontrak *canonical artifact* memisahkan orkestrator dari implementasi komponen, sedangkan *profile*, pipeline run, attempt, dan provenance digunakan untuk memilih konfigurasi serta mereproduksi dan memantau setiap proses. Eksperimen *one factor at a time* menggunakan sembilan *User Manual* Netgear (1.400 halaman, 200 pertanyaan bilingual) dan korpus peraturan pendidikan (256 dokumen, 25 pertanyaan). Pada $K = 30$, *structure-aware chunker* mencapai *recall* 0,740 pada *User Manual* dan *legal structure-aware* mencapai 0,6854 pada korpus hukum. *Dense indexer* BAAI/BGE-M3 mencapai *recall* 0,785 pada *User Manual* dan 0,6854 pada hukum, sedangkan *hybrid* memiliki MRR hukum tertinggi, 0,5439. Traversal *knowledge graph* hingga kedalaman dua mencapai *recall* lingkungan 1,0000 pada konfigurasi *sliding-window*. Hasil menunjukkan bahwa kontrak dan *plugin* memungkinkan strategi diganti secara terisolasi, tetapi konfigurasi terbaik tetap bergantung pada struktur dokumen dan tujuan metrik.

Index Terms—OMNI-RAG, basis pengetahuan, *plugin*, *chunking*, *indexing*, *knowledge graph*, RAG.

I. PENDAHULUAN

Sistem tanya jawab berbasis *Retrieval-Augmented Generation* (RAG) memerlukan sumber eksternal agar jawaban tidak hanya bergantung pada pengetahuan parametrik model bahasa [1, 2]. Kualitas sumber eksternal tersebut ditentukan jauh sebelum model menghasilkan jawaban: dokumen harus diubah menjadi unit yang dapat dicari, indeks harus memelihara identitas sumber, dan hubungan antardokumen perlu direpresentasikan ketika jawaban memerlukan konteks lintas dokumen. Tanpa ketiga hal tersebut, sistem dapat mengembalikan potongan yang benar secara leksikal tetapi salah konteks.

Kebutuhan tersebut menjadi lebih sulit pada proyek Capstone yang menggabungkan empat kontribusi teknis ke dalam satu platform. Proyek ini membangun OMNI-RAG, yaitu platform RAG multidomain untuk chatbot asisten hukum peraturan perundang-undangan Indonesia. Korpus hukum bersifat hierarkis: Bab, Bagian, Pasal, Ayat, Huruf, dan Angka mempunyai hubungan induk-anak yang memengaruhi makna ketentuan [3]. Di sisi lain, platform juga harus dapat memproses dokumen non-hukum seperti *User Manual* yang memiliki struktur

berbeda. Dengan demikian, satu strategi pemotongan atau pencarian tidak dapat diasumsikan cocok untuk semua sumber.

Paper ini menyajikan kontribusi individual pada pembentukan basis pengetahuan OMNI-RAG. Fokusnya adalah tiga komponen pada jalur offline, yaitu *chunker*, *indexer*, dan pembangun *knowledge graph*, serta mekanisme *plugin* dan *profile* yang mengikat ketiganya menjadi pipeline yang dapat dipertukarkan. Jalur offline menerima keluaran parser berupa dokumen kanonik, lalu menghasilkan unit pencarian, indeks, dan graf yang digunakan oleh komponen retrieval pada jalur online. Komponen akuisisi, antarmuka percakapan, dan lapisan aplikasi lain dipaparkan hanya sebagai konteks integrasi.

Pertanyaan penelitian paper ini adalah sebagai berikut.

- 1) Bagaimana kontrak artefak kanonik dan arsitektur *plugin* dapat memisahkan strategi pemotongan, pengindeksan, dan pembentukan graf dari orkestrator inti?
- 2) Bagaimana strategi *structure-aware* dibandingkan dengan strategi generik pada dokumen manual dan peraturan hukum?
- 3) Bagaimana sparse, dense, dan hybrid *indexer* memengaruhi kelengkapan dan urutan konteks yang ditemukan?
- 4) Sejauh mana graf hukum dapat memperluas lingkungan dokumen yang ditemukan tanpa mengubah metrik retrieval langsung?

Kontribusi paper ini meliputi: (1) rancangan kontrak `CanonicalDocument`, `CanonicalChunks`, `CanonicalIndex`, dan `CanonicalGraph` yang dipakai lintas domain; (2) implementasi dan evaluasi *plugin* untuk *chunker*, *indexer*, dan *knowledge graph* dengan pemilihan melalui *profile*; (3) prosedur profil, pipeline run, attempt, dan provenance untuk membandingkan eksperimen secara terisolasi; serta (4) analisis hasil pada dua korpus yang menunjukkan bahwa struktur dokumen dan tujuan metrik menentukan konfigurasi terbaik.

Ruang lingkup paper dibatasi pada konstruksi dan evaluasi basis pengetahuan. Pembahasan tidak mengklaim efektivitas seluruh chatbot atau kualitas sistem di luar konfigurasi eksperimen yang dilaporkan. Bagian selanjutnya menjelaskan konteks Capstone dan rancangan arsitektur, diikuti metode eksperimen, hasil, keterbatasan, dan kesimpulan.

II. KONTEKS CAPSTONE DAN RANCANGAN SISTEM

A. Cerita Besar Proyek

OMNI-RAG dikembangkan sebagai proyek Capstone oleh empat orang. Tujuan proyek besar tersebut adalah menyediakan chatbot asisten hukum yang dapat menelusuri peraturan perundang-undangan Indonesia, menghubungkan ketentuan yang saling berkaitan, dan mengembalikan dasar hukum yang dapat diverifikasi. Sistem lengkap terdiri atas jalur offline untuk membentuk basis pengetahuan dan jalur online untuk menggunakan indeks serta graf ketika menerima pertanyaan. Paper ini mengambil satu bagian dari rantai tersebut, yaitu pembangunan basis pengetahuan, karena kualitas komponen offline menjadi batas atas kualitas konteks yang dapat ditemukan pada tahap berikutnya.

Pembagian tanggung jawab tim tidak diperlakukan sebagai empat aplikasi yang terpisah. Setiap kontribusi bertemu pada artefak kanonik dan kontrak layanan yang sama. Tabel I merangkum hubungan antara kontribusi penulis dan kontribusi anggota tim lain.

B. Arsitektur Offline

Jalur offline mengikuti alur *raw document* menuju parser, dokumen kanonik, *chunker*, dan dua proyeksi yang dibangun secara paralel oleh *indexer* dan *knowledge graph*. Satu rangkaian tersebut dibungkus sebagai *profile* yang menentukan datasource, versi *plugin*, konfigurasi, dan kebijakan materialisasi. Gambar 1 menampilkan rancangan yang dipakai pada eksperimen.

C. Kontrak plugin dan profile

Arsitektur modular RAG memisahkan tahap transformasi sehingga strategi dapat diuji dan diganti tanpa mengubah seluruh alur [4]. Pada OMNI-RAG, setiap *plugin* memiliki identitas, tipe komponen, versi, entry point, kapabilitas, dan skema konfigurasi. *Plugin Registry* memvalidasi manifest tersebut sebelum orkestrator memuat implementasinya. Validasi ini mencegah komponen yang menghasilkan artefak tidak kompatibel dijalankan dalam pipeline yang sama.

profile adalah konfigurasi yang memilih satu implementasi untuk setiap tahap dan mengikat pipeline pada datasource tertentu. Contoh *profile* untuk manual memilih *generic-structure-aware* dan *dense-semantic*, sedangkan *profile* hukum memilih *legal-structure-aware* dan dapat mengaktifkan pembangun *knowledge graph*. Beberapa *profile* dapat menggunakan dokumen yang sama melalui fan-out, tetapi hasil indeks dan graf tetap diisolasi berdasarkan identitas *profile*.

Untuk mendukung reproducibility, setiap eksekusi menyimpan identitas datasource, *profile*, versi *plugin*, hash konfigurasi, waktu mulai dan selesai, jumlah masukan, jumlah keluaran, jumlah penolakan, status, serta pesan kesalahan. Satu pipeline run dapat memiliki beberapa attempt ketika materialisasi gagal atau konfigurasi perlu diulang. Pencatatan tersebut menjadi lapisan monitoring operasional: peneliti dapat membandingkan kondisi eksperimen tanpa mencampurkan keluaran dari run

berbeda dan operator dapat menemukan tahap yang gagal tanpa membentuk ulang semua artefak.

D. Artefak Kanonik

Kontrak data adalah inti integrasi. *CanonicalDocument* mempertahankan teks, blok berhierarki, lokasi halaman, metadata, dan provenance dari parser. *chunker* mengubahnya menjadi *CanonicalChunks* yang memiliki ID stabil, jenis unit, teks, parent, posisi, dan fingerprint. Artefak yang sama menjadi masukan bagi *indexer* dan *knowledge graph* sehingga keduanya merujuk sumber yang identik. Tabel II merangkum field dan konsumen utama.

E. Strategi chunker

chunker generik menyediakan tiga strategi dengan kontrak keluaran yang sama. *Sliding-window* menggunakan jendela berukuran tetap dan overlap. *Recursive* mencoba pemisah dari paragraf, kalimat, kata, hingga karakter ketika batas ukuran terlampaui. *Structure-aware* menggunakan outline parser, mempertahankan heading induk, dan baru melakukan pemecahan rekursif ketika bagian target terlalu panjang. Segmentasi berbasis struktur didukung oleh temuan bahwa batas semantik dapat mengurangi pemisahan informasi yang diperlukan bersama [5].

Untuk peraturan, *legal structure-aware* memusatkan unit pada Pasal dan membawa konteks Bab atau Bagian. Satu Pasal dapat menghasilkan *parent chunk* yang utuh dan beberapa *child chunk* yang lebih kecil. Child digunakan sebagai kandidat pencarian, sedangkan parent dipulihkan ketika konteks lengkap dibutuhkan. Pendekatan ini sesuai dengan struktur normatif yang menempatkan Pasal, Ayat, Huruf, dan Angka dalam hubungan hierarkis [3, 6].

F. Strategi indexer

indexer hanya memproses unit bertipe retrieval atau both, tetapi tetap menyimpan unit context untuk pemulihan parent dan sitasi. *Sparse indexer* menggunakan BM25 untuk mencocokkan istilah, sedangkan *dense indexer* memetakan kueri dan chunk ke ruang vektor dengan BAAI/BGE-M3. *Hybrid indexer* menggabungkan peringkat sparse dan dense dengan Reciprocal Rank Fusion. Perbedaan ketiga strategi ini diuji pada chunk dan datasource yang sama sehingga perubahan hasil dapat dikaitkan dengan cara pencarian, bukan dengan perubahan unit sumber [7, 8, 9].

G. Pembangunan knowledge graph

knowledge graph bersifat opsional dan menerima *CanonicalChunks* yang sama. Pada domain hukum, graf deterministik membentuk tipe simpul Regulation, RevisePasal, dan Pasal. Relasi internal HAS menjaga struktur dokumen, sedangkan relasi antardokumen mencakup AMENDS, ESTABLISHES, dan REVOKES. Setiap sisi menyimpan evidence dan provenance sehingga traversal tidak terlepas dari sumbernya.

Pada tahap online, hasil retrieval langsung menjadi seed. Resolver mengelompokkan seed berdasarkan regulasi, menelusuri relasi hingga kedalaman dua, memetakan tetangga kembali ke

TABLE I
PEMETAAN KONTRIBUSI PADA PROYEK CAPSTONE OMNI-RAG

Kelompok	Ruang tanggung jawab	Titik integrasi
Akuisisi dan ekstraksi	Sumber dokumen, parser, pemulihan struktur, dan provenance awal.	Menghasilkan <code>CanonicalDocument</code> yang menjadi masukan seragam bagi komponen pembentukan basis pengetahuan.
Pembentukan basis pengetahuan	<i>chunker</i> , <i>indexer</i> , pembangun <i>knowledge graph</i> , materialisasi proyeksi, dan pengujian strategi.	Menghasilkan <code>CanonicalChunks</code> , <code>CanonicalIndex</code> , dan <code>CanonicalGraph</code> ; bagian ini merupakan fokus paper.
Retrieval dan aplikasi	Pencarian sparse, dense, atau hybrid, pemulihan konteks induk, layanan percakapan, dan autentikasi.	Mengonsumsi indeks dan graf berdasarkan identitas artefak serta <i>profile</i> yang aktif.
Orkestrasi dan penjelasan	Resolusi rencana pemrosesan, dispatch, atribusi sumber, dan penyajian jejak proses.	Menggunakan metadata run, provenance, dan identitas hasil retrieval untuk menjelaskan konteks yang dipilih.

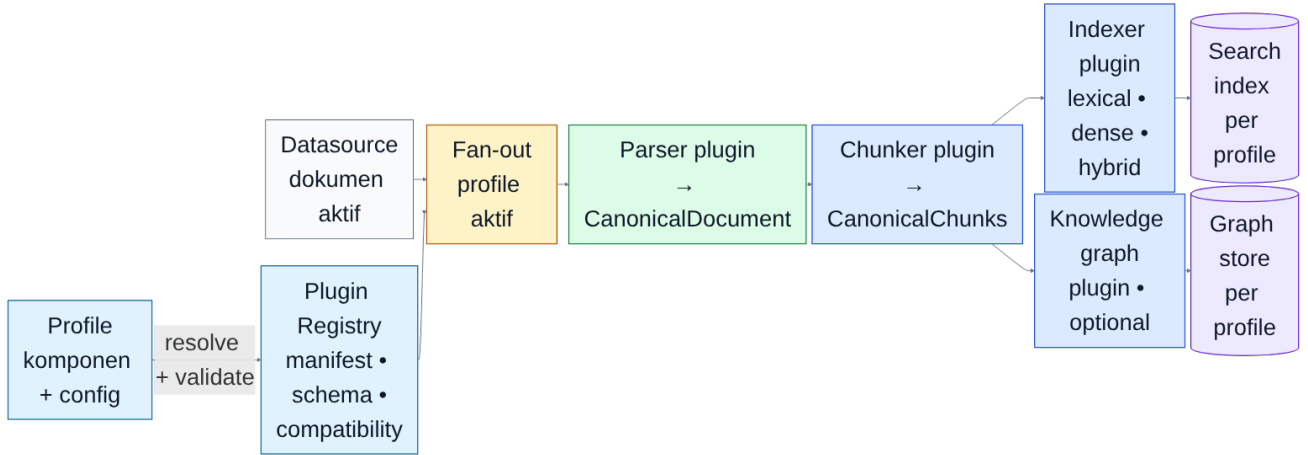


Fig. 1. Pipeline berbasis *plugin* dan artefak kanonik pada jalur pembentukan basis pengetahuan

TABLE II
KONTRAK ARTEFAK PADA PEMBENTUKAN BASIS PENGETAHUAN

Artefak	Isi minimal dan konsumen
<code>CanonicalDocument</code>	ID dokumen, blok berhierarki, teks, halaman, metadata, dan provenance; dikonsumsi <i>chunker</i> .
<code>CanonicalChunks</code>	ID chunk, tipe retrieval/context, teks, parent, posisi, metadata, dan fingerprint; dikonsumsi <i>indexer</i> dan <i>knowledge graph</i> .
<code>CanonicalIndex</code>	ID chunk, field leksikal atau vektor, filter metadata, versi model, dan provenance; dimaterialisasi ke OpenSearch.
<code>CanonicalGraph</code>	ID simpul dan sisi, endpoint, tipe relasi, evidence, scope, dan provenance; dimaterialisasi ke Neo4j.

ID chunk, lalu melakukan deduplikasi. Graf tidak menggantikan pencarian teks; graf memperluas lingkungan dokumen setelah kandidat awal ditemukan. Prinsip tersebut membedakan metrik recall langsung dari metrik jangkauan lingkungan graf, sebagaimana juga menjadi perhatian pada pendekatan GraphRAG [10].

III. METODE DAN RANCANGAN EKSPERIMEN

A. Lingkungan Implementasi

Pipeline dijalankan pada Python 3.13 dengan layanan penyimpanan metadata, object storage, antrean pesan, OpenSearch, dan Neo4j. OpenSearch digunakan untuk proyeksi sparse dan dense, sedangkan Neo4j digunakan untuk proyeksi graf. Embedding dibuat dengan BAAI/BGE-M3 karena model tersebut

TABLE III
KOMPONEN LINGKUNGAN EKSPERIMEN

Komponen	Konfigurasi
Runtime	Python 3.13, worker asinkron, Docker
Penyimpanan operasional	PostgreSQL dan object storage
Pencarian	OpenSearch 3.3.0 untuk lexical dan dense
Graf	Neo4j 5.26
Embedding	BAAI/BGE-M3
Protokol	OFAT, seed dan artefak dibekukan per run

mendukung masukan multibahasa dan panjang [9]. Seluruh konfigurasi memakai API, worker, model, dan jumlah kandidat yang sama ketika satu faktor sedang dibandingkan.

B. Korpus dan Pertanyaan

User Manual terdiri atas sembilan manual produk Netgear: CM2000, LM1200, R6350, R7000, RAX120, RAX50, RAXE500, RBK852, dan XR500. Total korpus berjumlah 1.400 halaman. Dari kandidat konteks yang disampling secara terstratifikasi berdasarkan dokumen dan posisi halaman, dibekukan 100 slot menggunakan seed 20260831. Setiap slot menghasilkan satu pertanyaan bahasa Inggris dan satu pertanyaan bahasa Indonesia sehingga terdapat 200 pertanyaan, tetapi dua pertanyaan dalam satu slot berbagi context reference

yang sama. Pertanyaan Indonesia menguji kemampuan model multibahasa pada dokumen sumber berbahasa Inggris.

Korpus hukum dibatasi pada peraturan bidang pendidikan Indonesia. Datasource eksperimen berisi 256 dokumen: 49 dokumen memuat context reference dan 207 dokumen noise yang dipilih secara terstratifikasi berdasarkan jenis, status, tahun, dan panjang. Dataset memiliki 25 pertanyaan yang disusun untuk eksperimen dan diverifikasi ahli, 148 baris reference, 144 konteks unik, dan 45 dokumen seed. Korpus dan pemetaan reference dibekukan sebelum perbandingan strategi.

C. Protokol Perbandingan

Eksperimen memakai pendekatan *one factor at a time* (OFAT). Pada kelompok *chunker*, datasource, parser, model embedding, dan *indexer* dibuat tetap; hanya strategi segmentasi yang diganti. Pada kelompok *indexer*, *chunker*, datasource, parser, dan embedding dibuat tetap; hanya strategi sparse, dense, atau hybrid yang diganti. Jalur retrieval langsung menonaktifkan traversal *knowledge graph* sehingga perbedaan hasil dapat dikaitkan dengan komponen yang sedang diuji. Eksperimen graf dijalankan terpisah pada graf hukum global.

Pada *User Manual*, *chunker* dibandingkan dengan batas 512 karakter. *Sliding-window* dan *recursive* memakai overlap 51 karakter, sedangkan *structure-aware* menggunakan target text block dan membawa parent context. Pada peraturan, *legal structure-aware* memusatkan pemotongan pada Pasal dan memakai parent hydration. Semua *indexer* dense menggunakan BAAI/BGE-M3, dan hybrid menggabungkan kanal lexical serta dense dengan RRF.

Cutoff kandidat disapu dari 5 hingga 60 dengan interval 5. $K = 30$ digunakan sebagai cutoff bersama karena menyedikan kandidat bagi tahap downstream tanpa meneruskan seluruh ekor ranking. Ketika dampak pada jawaban dibaca, sepuluh evidence teratas digunakan sebagai $K = 10$. Pemilihan konfigurasi didasarkan pada recall, sedangkan precision, MRR, dan NDCG dipakai sebagai diagnosis kepadatan dan posisi hasil. Perancangan ini mengikuti kebutuhan evaluasi lintas korpus: konfigurasi dan sumber harus dikunci sebelum hasil dibandingkan [11].

D. Metrik

Untuk pertanyaan q , himpunan context reference dinyatakan sebagai G_q , sedangkan referensi yang dicakup oleh K kandidat dinyatakan sebagai $C_q(K)$. Recall dihitung dengan

$$R_q @ K = \frac{|G_q \cap C_q(K)|}{|C_q(K)|}. \quad (1)$$

Precision menggunakan penyebut K , sementara MRR memberi bobot pada posisi reference pertama dan NDCG memberi bobot lebih besar pada hasil relevan di posisi awal. Nilai agregat adalah rata-rata seluruh pertanyaan pada korpus terkait.

Untuk graf, N_q adalah himpunan dokumen tetangga yang menjadi ground truth dan $H_q(K)$ adalah dokumen yang dicapai setelah seed retrieval dan traversal hingga kedalaman dua. Metrik jangkauan lingkungan dihitung dengan

$$R_q^K @ K = \frac{|N_q \cap H_q(K)|}{|H_q(K)|}. \quad (2)$$

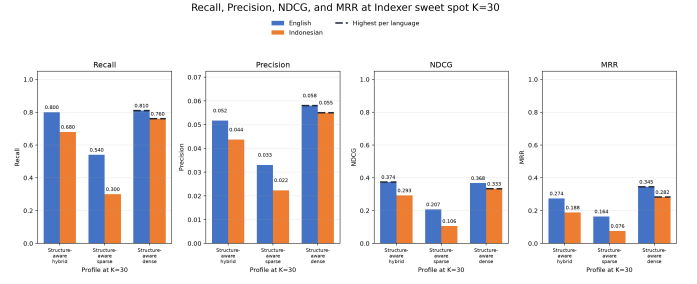


Fig. 2. Ringkasan metrik *indexer* pada *User Manual* dengan $K = 30$

Metrik ini tidak menggantikan recall context reference: satu dokumen yang ditemukan dapat membuka beberapa tetangga meskipun reference teksnya bukan kandidat pertama.

IV. HASIL DAN PEMBAHASAN

A. Dampak Pemeliharaan Struktur

Tabel V merangkum perbandingan *chunker* pada cutoff 30. Pada *User Manual*, *structure-aware* dengan hybrid *indexer* memperoleh recall 0,740, lebih tinggi 0,055 daripada *sliding-window* dan 0,115 daripada *recursive*. Pada korpus hukum, *legal structure-aware* dengan dense *indexer* memperoleh recall 0,6854, lebih tinggi 0,0509 daripada *sliding-window* dan 0,0124 daripada *recursive*.

Hasil tersebut menunjukkan bahwa batas struktur membawa informasi yang diperlukan bersama isi jawaban. Pada manual, parent context mempertahankan judul dan bagian prosedur; pada hukum, child chunk tetap dapat dicari tetapi parent Pasal tersedia ketika konteks dipulihkan. Keunggulan tidak muncul pada semua metrik: *sliding-window* memiliki MRR sedikit lebih tinggi pada manual dan NDCG sedikit lebih tinggi pada hukum. Hal ini memperlihatkan perbedaan antara menemukan banyak reference dan menempatkan satu hit pertama di posisi awal.

B. Dampak Jenis indexer

Tabel VI memperlihatkan bahwa dense menjadi pilihan terbaik ketika tujuan utama adalah kelengkapan reference. Pada *User Manual*, dense mencapai recall 0,785, precision 0,057, NDCG 0,350, dan MRR 0,314. Nilai tersebut mengungguli sparse dengan selisih recall 0,365. Pertanyaan Indonesia juga mengalami penurunan yang lebih kecil pada dense (recall 0,760) dibandingkan sparse (0,300), yang menunjukkan manfaat representasi multibahasa pada korpus manual berbahasa Inggris.

Pada peraturan, dense menghasilkan recall dan NDCG tertinggi, sedangkan hybrid menghasilkan MRR tertinggi. Sinyal lexical membantu menempatkan satu hit pertama pada posisi awal, tetapi tidak selalu menambah seluruh reference yang diperlukan dalam 30 kandidat. Oleh karena itu, pemilihan komponen harus menyatakan tujuan optimasi: dense dipilih untuk coverage, sedangkan hybrid dapat dipertimbangkan ketika hit pertama lebih penting.

TABLE IV
CAKUPAN KORPUS DAN UNIT EVALUASI

Korpus	Dokumen	Pertanyaan	Reference dan tujuan
User Manual	9 manual Netgear, 1.400 halaman	200, terdiri atas 100 pasangan Inggris-Indonesia	Satu context reference per slot; menguji pemeliharaan struktur dan retrieval lintas bahasa.
Peraturan hukum	256 dokumen, 49 dokumen reference dan 207 noise	25 pertanyaan, 148 baris reference dan 144 konteks unik	Menguji unit Pasal/Ayat, pencarian lintas peraturan, dan perluasan lingkungan graf.

TABLE V
HASIL PERBANDINGAN *CHUNKER* PADA CUTOFF $K = 30$

Korpus	Strategi	Recall	Precision	NDCG	MRR
User Manual	Structure-aware hybrid	0,740	0,048	0,333	0,231
User Manual	Recursive	0,625	0,032	0,291	0,216
User Manual	Sliding-window	0,685	0,041	0,322	0,232
Peraturan	Legal structure-aware dense	0,6854	0,1520	0,5040	0,5240
Peraturan	Recursive	0,6730	0,1453	0,4909	0,4937
Peraturan	Sliding-window	0,6345	0,1507	0,5042	0,4626

TABLE VI
HASIL PERBANDINGAN *INDEXER* PADA CUTOFF $K = 30$

Korpus	Strategi	Recall	Precision	NDCG	MRR
User Manual	Structure-aware dense	0,785	0,057	0,350	0,314
User Manual	Structure-aware hybrid	0,740	0,048	0,333	0,231
User Manual	Structure-aware sparse	0,420	0,028	0,157	0,120
Peraturan	Legal structure-aware dense	0,6854	0,1520	0,5040	0,5240
Peraturan	Structure-aware hybrid	0,6412	0,1307	0,4797	0,5439
Peraturan	Structure-aware sparse	0,6381	0,1253	0,4262	0,4613

TABLE VII
RECALL LINGKUNGAN *KNOWLEDGE GRAPH* HUKUM PADA $K = 30$

Konfigurasi seed	$R^{KG}@30$
Legal structure-aware dense	0,9556
Recursive	0,9975
Sliding-window	1,0000
Structure-aware sparse	0,9672
Structure-aware hybrid	0,9975

C. Perluasan melalui knowledge graph

Eksperimen graf menggunakan 17 relasi berarah dan 24 relasi tak berarah. Seed berasal dari kandidat retrieval langsung, kemudian traversal dilakukan sampai kedalaman dua. Tabel VII menunjukkan bahwa sliding-window mencapai jangkauan lingkungan penuh, 1,0000, sedangkan recursive dan hybrid mencapai 0,9975. Dense berada pada 0,9556 dan sparse pada 0,9672.

Nilai graf yang tinggi tidak berarti bahwa retrieval langsung sudah menemukan seluruh context reference. Graf mengukur apakah dokumen tetangga yang terhubung dapat dijangkau dari seed, sehingga satu seed yang tepat dapat membuka beberapa dokumen lain. Karena itu, *knowledge graph* diposisikan sebagai ekspansi konteks dan pemeriksaan hubungan, bukan pengganti indeks teks.

Knowledge-graph recall by configuration
Depth-two neighborhood coverage - seed documents excluded from the neighborhood

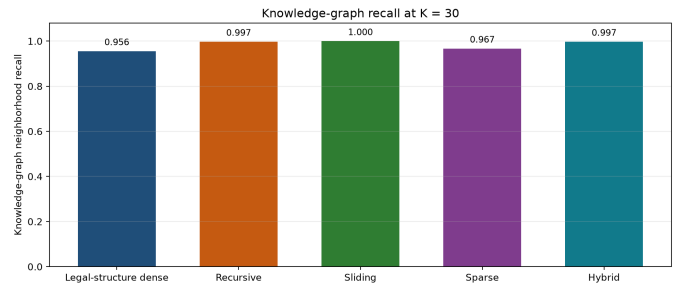


Fig. 3. Recall lingkungan graf untuk konfigurasi hukum pada $K = 30$

D. Dampak Hilir dan Monitoring

Sebagai pemeriksaan hilir, sepuluh evidence teratas dikirim ke model generatif. Pada *User Manual*, konfigurasi dense menghasilkan correctness 0,788 dan faithfulness 0,958. Pada korpus hukum, legal structure-aware dense menghasilkan correctness 0,690 dan faithfulness 0,972. Angka ini tidak digunakan untuk mengklaim kualitas chatbot secara keseluruhan; angka tersebut hanya menunjukkan bahwa coverage yang lebih baik pada tahap basis pengetahuan dapat diteruskan menjadi evidence yang lebih lengkap.

Monitoring run memberikan konteks untuk membaca hasil

tersebut. Setiap baris hasil dapat ditelusuri ke datasource, *profile*, versi *plugin*, konfigurasi, ID chunk, dan attempt materialisasi. Dengan demikian, perbedaan skor tidak berasal dari pencampuran indeks lama dan baru. Pencatatan status, jumlah keluaran, penolakan, durasi, dan kesalahan juga memungkinkan proses yang gagal diulang dari artefak terakhir yang valid. Paper ini mengevaluasi fungsi reproducibility tersebut secara kualitatif melalui keberhasilan pengulangan konfigurasi, bukan melalui benchmark latency produksi.

E. Pembahasan

Temuan utama dapat diringkas menjadi tiga implikasi. Pertama, struktur dokumen adalah bagian dari representasi basis pengetahuan, bukan sekadar metadata. Kedua, dense dan hybrid menjawab tujuan yang berbeda: dense unggul pada coverage, sedangkan hybrid dapat unggul pada hit pertama. Ketiga, *knowledge graph* memberi nilai ketika pertanyaan membutuhkan hubungan lintas dokumen, tetapi metrik graf harus dipisahkan dari metrik relevance teks.

Arsitektur *plugin* membuat temuan tersebut dapat diuji tanpa menulis ulang orkestrator. Strategi baru cukup mengimplementasikan kontrak yang sama, didaftarkan pada registry, lalu dipilih melalui *profile*. Keterbatasan arsitektur ini adalah kontrak tidak otomatis menjamin kualitas strategi: validasi skema dan provenance mencegah artefak rusak, tetapi pemilihan konfigurasi tetap memerlukan eksperimen pada domain dan pertanyaan yang relevan.

F. Keterbatasan

Eksperimen memiliki beberapa batasan. Korpus manual hanya mencakup sembilan produk dan pertanyaannya dibentuk dari konteks acuan, sedangkan korpus hukum hanya mewakili bidang pendidikan dan 25 pertanyaan. OFAT belum mengukur interaksi penuh antara semua kombinasi *chunker*, *indexer*, dan parameter traversal. Tidak ada klaim signifikansi statistik karena unit pertanyaan berpasangan dan sebagian reference berbagi sumber. Metrik graf juga mengukur jangkauan lingkungan, bukan validitas hukum atau kecukupan jawaban. Hasil sebaiknya dibaca sebagai bukti untuk konfigurasi yang diuji, bukan sebagai performa universal semua domain.

V. KESIMPULAN

Paper ini memisahkan kontribusi pembangunan basis pengetahuan dari keseluruhan cerita Capstone OMNI-RAG. Pada proyek empat orang tersebut, jalur offline menerima dokumen yang telah diparsing dan membentuk unit, indeks, serta graf yang dipakai bersama oleh aplikasi chatbot. Kontrak *CanonicalDocument*, *CanonicalChunks*, *CanonicalIndex*, dan *CanonicalGraph* membuat artefak dapat dipertukarkan, sedangkan *plugin*, *profile*, pipeline run, attempt, dan provenance menjaga ekstensibilitas serta keterulangan eksperimen.

Hasil menunjukkan bahwa *structure-aware chunking* meningkatkan coverage pada dua domain, tetapi strategi terbaik bergantung pada struktur sumber. Pada cutoff 30, konfigurasi *structure-aware* mencapai recall 0,740 pada *User*

Manual dan *legal structure-aware* mencapai 0,6854 pada peraturan. Dense indexer memperoleh recall tertinggi pada kedua korpus, yaitu 0,785 dan 0,6854. Hybrid menghasilkan MRR hukum tertinggi, 0,5439, sehingga coverage dan posisi hit pertama perlu diperlakukan sebagai tujuan yang berbeda. Traversal *knowledge graph* hingga kedalaman dua mencapai recall lingkungan 1,0000 pada konfigurasi sliding-window, tetapi metrik tersebut tidak menggantikan recall context reference.

Kontribusi teknis ini mendukung rancangan platform multidomain: komponen baru dapat ditambahkan melalui registry dan kontrak artefak tanpa mengubah orkestrator inti, sementara monitoring berbasis run dan provenance membuat hasil dapat diaudit. Pengembangan lanjutan perlu menguji kombinasi komponen secara factorial, memperluas korpus serta pertanyaan pengguna nyata, mengukur biaya dan latency, dan menambahkan benchmark kualitas relasi graf serta kompatibilitas versi *plugin*.

UCAPAN TERIMA KASIH

Penulis mengucapkan terima kasih kepada anggota tim Capstone, project owner, dan pembimbing yang memungkinkan pengembangan OMNI-RAG sebagai sistem terintegrasi. Paper ini menyajikan fokus kontribusi penulis pada pembentukan basis pengetahuan, dengan tetap menempatkan komponen tersebut dalam arsitektur tim secara utuh.

DAFTAR PUSTAKA

- [1] P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Apr. 12, 2021. arXiv: 2005.11401 [cs]. URL: <http://arxiv.org/abs/2005.11401> (visited on 12/24/2025). Pre-published.
- [2] Y. Huang and J. Huang. *A Survey on Retrieval-Augmented Text Generation for Large Language Models*. Aug. 23, 2024. arXiv: 2404.10981 [cs]. URL: <http://arxiv.org/abs/2404.10981> (visited on 12/24/2025). Pre-published.
- [3] Badan Pemeriksa Keuangan Republik Indonesia. *Undang-Undang Nomor 12 Tahun 2011 tentang Pembentukan Peraturan Perundang-Undangan*. JDIH BPK. Aug. 12, 2011. URL: <https://peraturan.bpk.go.id/Details/39188/uu-no-12-tahun-2011> (visited on 08/23/2026).
- [4] Y. Gao, Y. Xiong, M. Wang, and H. Wang. “Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks”. In: *arXiv* (July 26, 2024). arXiv: 2407.21059.
- [5] A. Jain, P. Aggarwal, and A. Saladi. “AutoChunker: Structured Text Chunking and Its Evaluation”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*. Association for Computational Linguistics, July 2025, pp. 983–995.

- [6] P. Akarajadwong et al. *NitiBench: A Comprehensive Study of LLM Framework Capabilities for Thai Legal Question Answering*. Aug. 21, 2025. arXiv: 2502.10868 [cs]. URL: <http://arxiv.org/abs/2502.10868> (visited on 12/24/2025). Pre-published.
- [7] S. Robertson and H. Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *FNT in Information Retrieval* 3.4 (2009), pp. 333–389.
- [8] V. Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Nov. 2020, pp. 6769–6781. arXiv: 2004.04906.
- [9] J. Chen et al. “M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Aug. 2024, pp. 2318–2335.
- [10] H. Han et al. *Retrieval-Augmented Generation with Graphs (GraphRAG)*. Jan. 8, 2025. arXiv: 2501.00309 [cs]. URL: <http://arxiv.org/abs/2501.00309> (visited on 12/22/2025). Pre-published.
- [11] N. Thakur, N. Reimers, J. Daxenberger, and I. Gurevych. “BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models”. In: *Proceedings of the 35th Conference on Neural Information Processing Systems Track on Datasets and Benchmarks*. Neural Information Processing Systems Foundation, 2021.